

SOLID

Consiste em um conjunto de cinco princípios fundamentais para a programação orientada a objetos, visando um código mais flexível, escalável e fácil de manter.

Os princípios são:

1. **Single Responsibility Principle** (SRP) – Cada classe deve ter apenas uma razão para mudar.
2. **Open-Closed Principle** (OCP) – Classes devem estar abertas para extensão, mas fechadas para modificação.
3. **Liskov Substitution Principle** (LSP) – Subtipos devem ser substituíveis por seus tipos base sem alterar o comportamento esperado do sistema.
4. **Interface Segregation Principle** (ISP) – Interfaces específicas devem ser preferidas a interfaces genéricas.
5. **Dependency Inversion Principle** (DIP) – Os módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações.

Calistenia de Objetos

Trata-se de um conjunto de princípios de programação projetados para melhorar a qualidade do código, sua manutenibilidade e legibilidade através de uma série de nove regras.

Visão Geral

O termo combina "Objeto," referindo-se à Programação Orientada a Objetos, e "Calistenia," que significa exercícios destinados a melhorar a forma física. Assim, refere-se a exercícios que aprimoram habilidades e práticas de codificação.

As 9 Regras da Calistenia de Objetos

1. **Apenas Um Nível de Endentação por Método:** Esta regra incentiva a simplicidade nos métodos. Se um método tem vários níveis de endentação, provavelmente faz demais e deve ser refatorado em métodos menores.
2. **Não Use a Palavra-chave ELSE:** Evitar afirmações else pode levar a um código mais claro. Em vez disso, considere usar polimorfismo ou cláusulas de guarda para lidar com condições sem aninhamento.

3. **Envolva Todos os Primitivos e Strings em Classes:** Isso promove uma melhor encapsulação e torna o código mais compreensível ao criar abstrações significativas em torno de tipos primitivos e strings.
4. **Coleções de Primeiro Nível:** Em vez de usar arrays ou coleções diretamente, crie classes que encapsulem coleções, fornecendo métodos para manipulá-las. Isso melhora a legibilidade e a manutenção.
5. **Um Ponto Por Linha:** Esta regra sugere que cada linha de código deve chamar apenas um método. Esta prática melhora a legibilidade e torna mais fácil entender o fluxo do programa.
6. **Não Abrevie:** Use nomes completos para variáveis e métodos para aumentar a clareza. Abreviações podem levar à confusão e reduzir a legibilidade do código.
7. **Mantenha Todas as Classes Com Menos de 50 Linhas:** Classes menores são mais fáceis de entender e manter. Se uma classe exceder esse limite, considere dividi-la em classes menores e mais focadas.
8. **Não tenha Classes com Mais de Duas Variáveis de Instância:** Esta regra incentiva a simplicidade no design de classes. Se uma classe tiver mais de duas variáveis de instância, ela pode estar fazendo demais e deve ser refatorado.
9. **Sem Getters ou Setters:** Este princípio promove a encapsulação ao desencorajar o acesso direto ao estado de um objeto. Em vez disso, use métodos que operem nos dados do objeto, respeitando o princípio 'Diga, Não Pergunte'.

Coesão x Acoplamento

Coesão: módulos com uma funcionalidade <-> Coesão forte.

Acoplamento: baixa interdependência entre módulo <-> acoplamento fraco